Planetaire Mono

B612 letterforms
Hack infrastructure
Nerd Font icons

Joshua Levy
github.com/jlevy/planetaire
Version 0.1.0

Planetaire Mono is a composite monospace font that merges the highly legible letterforms of B612, a typeface designed by Intactile Design for Airbus cockpit displays, with Hack Nerd Font's complete infrastructure: punctuation, symbols, and 12,000+ developer icons.

The result is a font optimized for terminal and editor use that combines aviation-grade character clarity with full programming language coverage. The name is a nod to asteroid B-612 from *The Little Prince*.

[illegible]
[illegible]
[illegible]
Medium [illegible]
Medium Italic [illegible]
[illegible] (700)
[illegible] (700)
[illegible]
[illegible]

[illegible]
[illegible]
[illegible]
[illegible]
[illegible]
[illegible]

Planetaire Text Sample

The quick brown fox jumps over the lazy dog. Pack my box with five dozen liquor jugs. How vexingly quick daft zebras jump!

In the great void between stars, instruments must be read without error. Every glyph must be unambiguous: the digit 0 distinct from the letter O, the numeral 1 clearly not a lowercase l or uppercase I. B612 was born from this requirement. Originally designed for cockpit displays at Airbus, where a misread character could mean the difference between FL350 and FL850. Planetaire Mono inherits that precision and pairs it with the full symbol coverage a programmer needs: braces, brackets, pipes, arrows, and 12,000 icons ready for a modern terminal.

FRENCH · GERMAN · SPANISH · TURKISH

Les naïfs ægithales hâtifs pondsjflam au zéphyr joyeux. Falsches Üben von Xylophonmusik quält jeden größeren Zwerg. El veloz murciélago hindú comía feliz cardillo y kiwi. Pijamalı hasta yağız şoföre çabucak güvendi.

GREEK

Ξεσκεπάζω τὴν ψυχοφθόρα βδελυγμία.

CYRILLIC

Съешь же ещё этих мягких французских булок, да выпей чаю. Широкая электрификация южных губерний даст мощный толчок подъёму сельского хозяйства.

Planetaire Iconic Texts

Alan Turing · “Computing Machinery and Intelligence” (1950)

I propose to consider the question, “Can machines think?” This should begin with definitions of the meaning of the terms “machine” and “think.” The definitions might be framed so as to reflect so far as possible the normal use of the words, but this attitude is dangerous. If the meaning of the words “machine” and “think” are to be found by examining how they are commonly used it is difficult to escape the conclusion that the meaning and the answer to the question, “Can machines think?” is to be sought in a statistical survey such as a Gallup poll. But this is absurd. Instead of attempting such a definition I shall replace the question by another, which is closely related to it and is expressed in relatively unambiguous words.

[illegible] 7 [illegible]il [illegible]

[illegible]

[illegible]

[illegible] BB& [illegible]

[illegible] Bill [illegible]all [illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible] i.[illegible]., [illegible]

Planetaire Code Specimen: microGPT

```python
"""
The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

@karpathy
"""

import os       # os.path.exists
import math     # math.log, math.exp
import random   # random.seed, random.choices, random.gauss, random.shuffle
random.seed(42) # Let there be order among chaos

# Let there be a Dataset `docs`: list[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
random.shuffle(docs)
print(f"num docs: {len(docs)}")

# Let there be a Tokenizer to translate strings to sequences of integers ("tokens") and back
uchars = sorted(set(''.join(docs))) # unique characters in the dataset become token ids 0..n-1
BOS = len(uchars) # token id for a special Beginning of Sequence (BOS) token
vocab_size = len(uchars) + 1 # total number of unique tokens, +1 is for BOS
print(f"vocab size: {vocab_size}")

# Let there be Autograd to recursively apply the chain rule through a computation graph
class Value:
    __slots__ = ('data', 'grad', '_children', '_local_grads') # Python optimization for memory usage

    def __init__(self, data, children=(), local_grads=()):
        self.data = data                # scalar value of this node calculated during forward pass
        self.grad = 0                   # derivative of the loss w.r.t. this node, calculated in backward pass
        self._children = children       # children of this node in the computation graph
        self._local_grads = local_grads # local derivative of this node w.r.t. its children

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data + other.data, (self, other), (1, 1))

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data * other.data, (self, other), (other.data, self.data))

    def __pow__(self, other): return Value(self.data**other, (self,), (other * self.data**(other-1),))
    def log(self): return Value(math.log(self.data), (self,), (1/self.data,))
    def exp(self): return Value(math.exp(self.data), (self,), (math.exp(self.data),))
    def relu(self): return Value(max(0, self.data), (self,), (float(self.data > 0),))
    def __neg__(self): return self * -1
    def __radd__(self, other): return self + other
    def __sub__(self, other): return self + (-other)
    def __rsub__(self, other): return other + (-self)
    def __rmul__(self, other): return self * other
    def __truediv__(self, other): return self * other**-1
    def __rtruediv__(self, other): return other * self**-1

    def backward(self):
        topo = []
        visited = set()
        def build_topo(v):
            if v not in visited:
                visited.add(v)
                for child in v._children:
                    build_topo(child)
                topo.append(v)
        build_topo(self)
        self.grad = 1
        for v in reversed(topo):
            for child, local_grad in zip(v._children, v._local_grads):
                child.grad += local_grad * v.grad
```

```python
# Initialize the parameters, to store the knowledge of the model
n_layer = 1     # depth of the transformer neural network (number of layers)
n_embd = 16     # width of the network (embedding dimension)
block_size = 16 # maximum context length of the attention window (note: the longest name is 15 characters)
n_head = 4      # number of attention heads
head_dim = n_embd // n_head # derived dimension of each head
matrix = lambda nout, nin, std=0.08: [[Value(random.gauss(0, std)) for _ in range(nin)] for _ in range(nout)]
state_dict = {'wte': matrix(vocab_size, n_embd), 'wpe': matrix(block_size, n_embd), 'lm_head': matrix(vocab_size,
n_embd)}
for i in range(n_layer):
    state_dict[f'layer{i}.attn_wq'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wk'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wv'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wo'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.mlp_fc1'] = matrix(4 * n_embd, n_embd)
    state_dict[f'layer{i}.mlp_fc2'] = matrix(n_embd, 4 * n_embd)
params = [p for mat in state_dict.values() for row in mat for p in row] # flatten params into a single list[Value]
print(f"num params: {len(params)}")

# Define the model architecture: a function mapping tokens and parameters to logits over what comes next
# Follow GPT-2, blessed among the GPTs, with minor differences: layernorm -> rmsnorm, no biases, GeLU -> ReLU
def linear(x, w):
    return [sum(wi * xi for wi, xi in zip(wo, x)) for wo in w]

def softmax(logits):
    max_val = max(val.data for val in logits)
    exps = [(val - max_val).exp() for val in logits]
    total = sum(exps)
    return [e / total for e in exps]

def rmsnorm(x):
    ms = sum(xi * xi for xi in x) / len(x)
    scale = (ms + 1e-5) ** -0.5
    return [xi * scale for xi in x]

def gpt(token_id, pos_id, keys, values):
    tok_emb = state_dict['wte'][token_id] # token embedding
    pos_emb = state_dict['wpe'][pos_id] # position embedding
    x = [t + p for t, p in zip(tok_emb, pos_emb)] # joint token and position embedding
    x = rmsnorm(x) # note: not redundant due to backward pass via the residual connection

    for li in range(n_layer):
        # 1) Multi-head Attention block
        x_residual = x
        x = rmsnorm(x)
        q = linear(x, state_dict[f'layer{li}.attn_wq'])
        k = linear(x, state_dict[f'layer{li}.attn_wk'])
        v = linear(x, state_dict[f'layer{li}.attn_wv'])
        keys[li].append(k)
        values[li].append(v)
        x_attn = []
        for h in range(n_head):
            hs = h * head_dim
            q_h = q[hs:hs+head_dim]
            k_h = [ki[hs:hs+head_dim] for ki in keys[li]]
            v_h = [vi[hs:hs+head_dim] for vi in values[li]]
            attn_logits = [sum(q_h[j] * k_h[t][j] for j in range(head_dim)) / head_dim**0.5 for t in range(len(k_h))]
            attn_weights = softmax(attn_logits)
            head_out = [sum(attn_weights[t] * v_h[t][j] for t in range(len(v_h))) for j in range(head_dim)]
            x_attn.extend(head_out)
        x = linear(x_attn, state_dict[f'layer{li}.attn_wo'])
        x = [a + b for a, b in zip(x, x_residual)]
        # 2) MLP block
        x_residual = x
        x = rmsnorm(x)
        x = linear(x, state_dict[f'layer{li}.mlp_fc1'])
        x = [xi.relu() for xi in x]
        x = linear(x, state_dict[f'layer{li}.mlp_fc2'])
        x = [a + b for a, b in zip(x, x_residual)]

    logits = linear(x, state_dict['lm_head'])
    return logits

# Let there be Adam, the blessed optimizer and its buffers
learning_rate, beta1, beta2, eps_adam = 0.01, 0.85, 0.99, 1e-8
m = [0.0] * len(params) # first moment buffer
v = [0.0] * len(params) # second moment buffer

# Repeat in sequence
num_steps = 1000 # number of training steps
for step in range(num_steps):

    # Take single document, tokenize it, surround it with BOS special token on both sides
```

```python
    doc = docs[step % len(docs)]
    tokens = [BOS] + [uchars.index(ch) for ch in doc] + [BOS]
    n = min(block_size, len(tokens) - 1)

    # Forward the token sequence through the model, building up the computation graph all the way to the loss
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    losses = []
    for pos_id in range(n):
        token_id, target_id = tokens[pos_id], tokens[pos_id + 1]
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax(logits)
        loss_t = -probs[target_id].log()
        losses.append(loss_t)
    loss = (1 / n) * sum(losses) # final average loss over the document sequence. May yours be low.

    # Backward the loss, calculating the gradients with respect to all model parameters
    loss.backward()

    # Adam optimizer update: update the model parameters based on the corresponding gradients
    lr_t = learning_rate * (1 - step / num_steps) # linear learning rate decay
    for i, p in enumerate(params):
        m[i] = beta1 * m[i] + (1 - beta1) * p.grad
        v[i] = beta2 * v[i] + (1 - beta2) * p.grad ** 2
        m_hat = m[i] / (1 - beta1 ** (step + 1))
        v_hat = v[i] / (1 - beta2 ** (step + 1))
        p.data -= lr_t * m_hat / (v_hat ** 0.5 + eps_adam)
        p.grad = 0

    print(f"step {step+1:4d} / {num_steps:4d} | loss {loss.data:.4f}", end='\r')

# Inference: may the model babble back to us
temperature = 0.5 # in (0, 1], control the "creativity" of generated text, low to high
print("\n--- inference (new, hallucinated names) ---")
for sample_idx in range(20):
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    token_id = BOS
    sample = []
    for pos_id in range(block_size):
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax([l / temperature for l in logits])
        token_id = random.choices(range(vocab_size), weights=[p.data for p in probs])[0]
        if token_id == BOS:
            break
        sample.append(uchars[token_id])
    print(f"sample {sample_idx+1:2d}: {''.join(sample)}")
```

Planetaire Terminal

```python
def analyze_trajectory(altitude: float, velocity: float) -> dict:
    """Calculate orbital parameters."""

    G = 6.674e-11  # gravitational constant
    M = 5.972e24

    if altitude > 400_000:
        orbit_type = "LEO"
    elif altitude > 35_786_000:
        orbit_type = "GEO"

    period = 2 * math.pi * math.sqrt(altitude**3 / (G * M))

    return {"type": orbit_type, "period": period, "v": velocity}
```

Planetaire Character Set

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

a b c d e f g h i j k l m n o p q r s t u v w x y z

0 1 2 3 4 5 6 7 8 9

! “ # $ % & ‘ ( ) * + , - . / : ; < = > ? @ [\ ] ^ _
` { | } ~

À Á Â Ã Ä Å Æ Ç È É Ê Ë Ì Í Î Ï Ð Ñ Ò Ó Ô Õ Ö Ø Ù Ú Û Ü Ý Þ ß

à á â ã ä å æ ç è é ê ë ì í î ï ð ñ ò ó ô õ ö ø ù ú û ü ý þ ÿ

Ā ā Ă ă Ą ą Ć ć Ĉ ĉ Ċ ċ Č č Ď ď Đ đ Ē ē Ĕ ĕ Ė ė Ę ę Ě ě

Ğ ğ Ġ ġ Ģ ģ Ĥ ĥ Ħ ħ Ĩ ĩ Ī ī Ĭ ĭ Į į İ ı Ĳ ĳ Ĵ ĵ Ķ ķ

Α Β Γ Δ Ε Ζ Η Θ Ι Κ Λ Μ Ν Ξ Ο Π Ρ Σ Τ Υ Φ Χ Ψ Ω

α β γ δ ε ζ η θ ι κ λ μ ν ξ ο π ρ σ τ υ φ χ ψ ω

АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ

абвгдежзийклмнопрстуфхцчшщъыьэюя

Planetaire Weight Comparison

Regular (400)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

Italic (400)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

Medium (500)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

Medium Italic (500)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

Bold (700)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

Bold Italic (700)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

ExtraBold (800)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

ExtraBold Italic (800)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

Size comparison: Regular at various sizes

The quick brown fox jumps over the lazy dog

The quick brown fox jumps over the lazy dog

The quick brown fox jumps over the lazy dog

The quick brown fox jumps over the lazy dog

Planetaire Legibility

[illegible]

I l 1 | [illegible]

O 0 o [illegible]

r n m [illegible]

5 S 8 B [illegible]

2 Z 6 G [illegible]

; : [illegible]

( ) { } [ ] [illegible]

- ‐ — [illegible]

“ ” " [illegible]

‘ ’ ' ` [illegible]

[illegible]

[illegible]

0 0 O o

FL350 FL850 10.0.0.1

[illegible]

0 0 O o

FL350 FL850 10.0.0.1

[illegible]

( ) { } [ ] < > « » ‹ ›

[illegible]

+ − * / = ≠ ≤ ≥ ± × ÷ → ← ↑ ↓

Planetaire Symbols and Special Characters

. , ; : ! ? - - — ( ) [ ] { } / \ @ # $ % ^ & *
_ + = ~ `

" ' ` “ ” ‘ ’ « » ‹ ›

§ ¶ © ® ™ ° · … † ‡ ¤ ¢ £ ¥ € ¬ ¦

/ \ | ¦ ⁄ ∕

─ │ ┌ ┐ └ ┘ ├ ┤ ┬ ┴ ┼ ═ ║ ╔ ╗ ╚ ╝ ╠ ╣ ╦ ╩ ╬ ▀ ▄ █ ▌ ▐
░ ▒ ▓

Nerd Font Icons

Planetaire Provenance & License

Source Fonts

B612 Mono	Designed by Intactile Design for Airbus. Optimized for legibility in cockpit displays. Planetaire Mono takes its letters (A-Z, a-z), digits 0-9, and extended Latin/Greek/Cyrillic glyphs from B612. The zero glyph receives a center dot in post-processing for O/0 disambiguation, with circle (default) and rectangle (ss01) variants.
Hack	Chris Simpkins’ typeface designed for source code. Provides the base font structure: punctuation, symbols, metrics, and Nerd Font integration.
Nerd Fonts	Ryan McIntyre’s icon patching project. 12,000+ developer icons including Powerline, Font Awesome, Devicons, Material Design, and more, all included via the Hack Nerd Font base.

Build Pipeline

Planetaire Mono is built with a custom Python pipeline using fontTools. For each weight variant, the pipeline loads Hack Nerd Font as the base, merges B612 letter and digit glyphs by Unicode range, adds a center dot to B612’s zero for O/0 disambiguation, renames the result, and applies post-processing fixes. Medium weights are generated from Regular, and ExtraBold from Bold, via FontForge emboldening.

License

Planetaire Mono is released under the **SIL Open Font License 1.1 (OFL-1.1)**.

The OFL allows free use, modification, and redistribution of the font, including in commercial products, provided that modified versions are not sold by themselves and carry a different name.

The constituent fonts carry the following licenses:
- **B612 Mono**: SIL Open Font License 1.1 + Eclipse Public License 2.0
- **Hack**: MIT License
- **Nerd Fonts** patches: MIT License